

[i]NSTANT



Pointers

Presented By : Mazen Othman

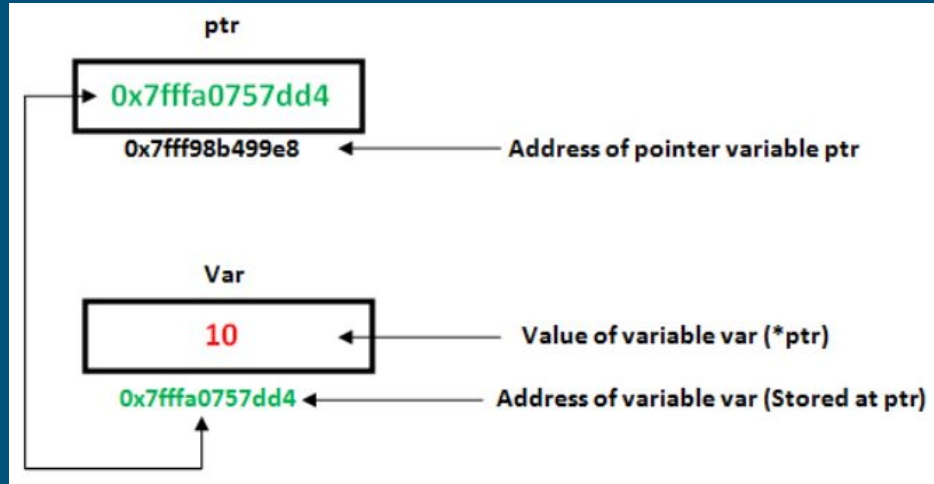
- Problem Solving Instructor



-

Variables & Pointers

- **Variable** : Container That Stores Certain Values.
- **Pointer** : Container That Stores An Address (Memory Location).
 - It's A Variable That Stores That Address Of Another Variable.



Pointers Operators

- To Get The Address Of Any Variable , You Can Put The Reference Operator (&) before The Variable – For Example :

```
// The Code
```

```
int x = 5 ;  
cout << &x ;
```

```
// The OutPut
```

```
// Note That The Output is Random Address , So if You Run  
The Same Code The Address Not Require To be The Same Address.  
0xe89b7ffd7c
```

Pointers Operators

- To Access The Value Of The Variable Using His Address , You Can Put The Asterisk Operator (*) before The Pointer – For Example :



```
// The Code
int x = 5 ;
int *ptr = &x ;
cout << *ptr ;
```

```
// The OutPut
5
```

Brainstorm With Me !



```
int x = 5 ;  
int *ptr = &x ;  
*ptr += 5 ;  
cout << x ;
```

What Is OutPut ?

Pointers & Variables

- To Create a Pointer For an Integer Value (Variable of Type int), The type of data whose address is stored inside the pointer must be specified.

For Example :

```
int x = 5 ; // --> Create A Variable named x
int *ptr ; // --> Create A Pointer for int Variable named ptr
ptr = &x ; // Store The Address Of Variable x in Pointer ptr
```

```
// The int Reserves 4 Bytes
```

```
0x5000
0x5001
0x5002
0x5003
```

Some Notes

- Each Data Type Has Its Own Pointer Variable , For int Variable We Use int Pointer and So on.
- It's Not Permit To Assign Values To The Pointer Using The Contents of Operator (*) Without having an Address.

Let's Drive in Some Examples :

Brainstorm With Me !

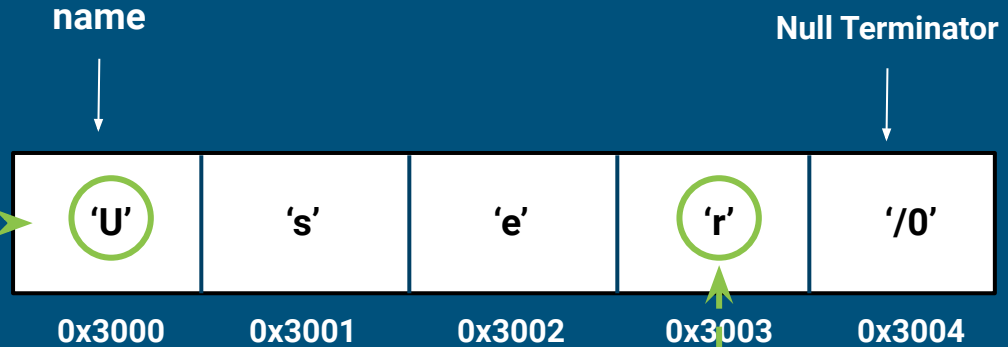


```
int * ptr ;  
*ptr = 10 ;  
cout << *ptr ;
```

What Is OutPut ?

Pointers With Array

- `char name [] = "User";`
`char *ptr = name;`
`cout << *ptr;`
`*ptr = 'A';`
`ptr += 3;`
`*ptr = 'm';`



Brainstorm With Me !



```
char name[] = "Mazen" ;  
char *ptr = name ;  
cout << *(ptr++) <<endl;  
cout << *(++ptr) <<endl;
```

What Is OutPut ?

Pointers With Functions

- Passing Parameters With to a Function With One Of Three Ways :
 - By Value : `void func ( int x ) {}`
 - By Reference : `void func ( int &x ) {}`
 - By Pointer : `void func ( int *x ) {}`

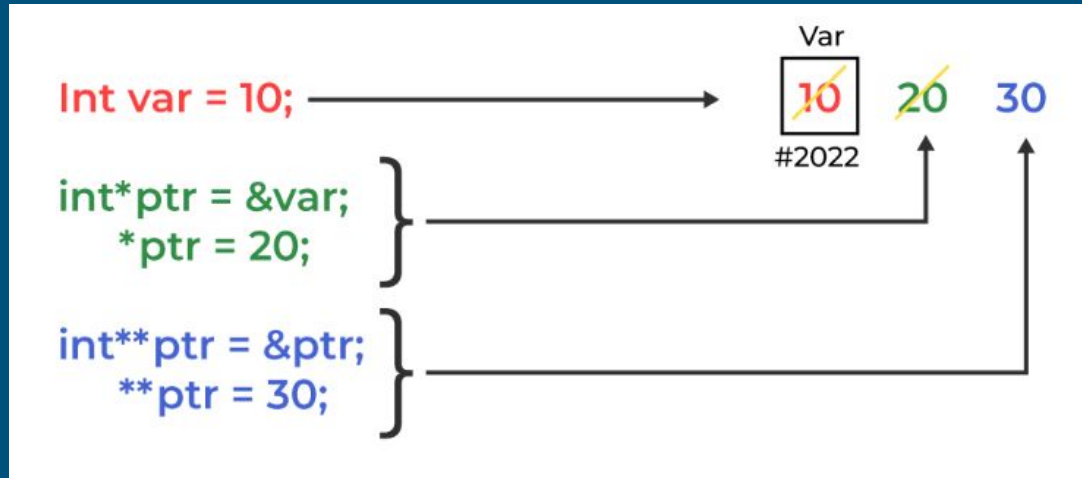
Brainstorm With Me !

```
void Func (int *x , int *y){  
    *x += 10 ;  
    y+= 10 ;  
}  
  
int main() {  
  
    int x = 5 , y = 5 ;  
    int *ptr1 = &x , *ptr2 = &y ;  
    Func( ptr1 , ptr2 ) ;  
    cout << x << " " << y ;  
}
```

What Is OutPut ?

Pointers To Pointers

- A pointer is a variable that stores the memory address of another variable.
- A pointer to pointer is a pointer that holds the address of another pointer.



How Pointers Work

Brainstorm With Me !



```
int x = 5 ;  
int *ptr = &x ;  
int **ptr2 = &ptr ;  
**ptr2 += 10 ;  
cout << ptr2 <<endl ;  
cout << *ptr2 <<endl ;  
cout << **ptr2 <<endl ;
```

What Is OutPut ?

Brainstorm With Me !



```
int x = 5 ;  
int *ptr = &x ;  
int **ptr2 = &ptr ;  
*ptr2 += 10 ;  
cout << ptr2 <<endl ;  
cout << *ptr2 <<endl ;  
cout << **ptr2 <<endl ;
```

What Is OutPut ?

Pointers With Loops

- Pointers provide a powerful way to iterate over arrays and strings.
- More efficient memory access and manipulation compared to index-based loops.
- Example :



```
int arr[] = {10, 20, 30, 40, 50};  
int *ptr = arr;  
for (int i = 0; i < 5; i++) {  
    cout << *(ptr+i);  
}
```

Memory Allocation [Important Points]

- Memory Structure
- Static Memory Allocation & Dynamic Memory Allocation
- Compile Time & RunTime
- Primitive Data Types & Non-Primitive Data Types
- Stack & Heap
- Activation Records in Stack - (Code / Functions) - Memory
- VLA's (Variable Length Array) & GCC Compiler
- New & delete KeyWords - Memory Leaks
- Malloc & Calloc & ReAlloc

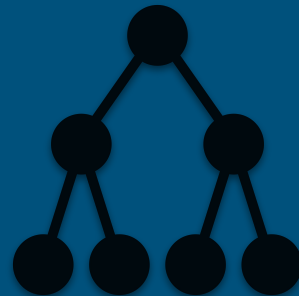


Any Questions

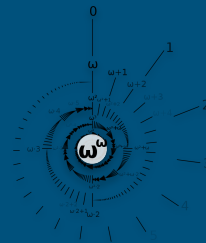




CODEFORCES



Thank
You



LeetCode

